

Mathematical Modeling and Closed-Loop PID Control

Four-DOF RRRR Desktop Manipulator with Positional Gripper

ROS 2 Humble implementation: `manipulator_ws`

September 2026

Abstract

This document derives the kinematic and numerical dynamic models implemented in the ROS 2 workspace, then derives the discrete PID controllers used by the closed-loop simulator. It distinguishes model parameters from experimentally identified hardware data, defines every coordinate conversion and ROS interface, and gives a direct path from simulated encoder feedback to encoder-based hardware deployment. The implementation referenced throughout is the PID simulation node; geometry and inverse kinematics are shared with the manipulator control package.

Scope. The current closed-loop simulator is a deliberately compact numerical plant, not a full multibody or finite-element model. It includes nominal inertia, viscous damping, gravity, effort saturation, hard stops, disturbances, and simulated encoder states. Parameters marked *nominal* must be identified on the real arm before gains are transferred to hardware.

Contents

1	Robot definition	2
1.1	Generalized coordinates	2
1.2	Geometry	2
1.3	Masses and URDF inertias	2
1.4	Limits and command space	3
2	Coordinate conventions and forward kinematics	3
2.1	Hardware-to-URDF conversion	3
2.2	Planar forward kinematics	3
3	Inverse kinematics	4
4	Numerical dynamic plant	4
4.1	Model structure	4
4.2	Gravity model	5
4.3	Discrete integration and stops	5
5	PID controller derivation	5
5.1	Position loops for J2–J4	5

5.2	Discrete integral and anti-windup	6
5.3	Base velocity loop	6
5.4	Gripper loop	6
5.5	Controller gains	6
6	Closed-loop interpretation	7
7	ROS 2 signal model	7
8	Numerical validation results	7
9	Encoder-based hardware deployment	8
9.1	Required measurements	8
9.2	Controller migration	8
9.3	Important limitation of hobby positional servos	8
9.4	Identification and tuning sequence	9
10	Model limitations and interpretation	9
11	Source-to-equation traceability	10

1 Robot definition

1.1 Generalized coordinates

The robot has one base-yaw joint and three serial pitch joints:

$$\mathbf{q} = \begin{bmatrix} q_1 & q_2 & q_3 & q_4 \end{bmatrix}^T.$$

The URDF convention used by the numerical plant is:

- q_1 : rotation about world $+z$; continuous.
- q_2, q_3, q_4 : rotation about each local $+y$ axis.
- At $q_2 = q_3 = q_4 = 0$, all arm links point along $+z$.
- Positive pitch rotates a link from $+z$ toward the radial $+x$ direction.

The gripper has two mirrored prismatic joints. Both receive the same joint coordinate; opposite URDF axis vectors create symmetric finger motion.

1.2 Geometry

Table 1: Geometry used by URDF, FK, IK, and PID simulation.

Symbol	Meaning	Value
h_0	shoulder height above ground	0.090 m
L_1	shoulder to elbow	0.100 m
L_2	elbow to wrist	0.090 m
L_3	wrist to end-effector tip	0.160 m
r_b	base radius	0.060 m
h_b	base-cylinder height	0.070 m
w_l, d_l	link cross-section	0.030×0.030 m
s_f	finger travel per side	0.030 m

The nominal geometric reach is

$$L_1 + L_2 + L_3 = 0.350 \text{ m},$$

but the usable workspace is smaller and non-convex because of joint limits, ground clearance, and end-effector orientation feasibility.

1.3 Masses and URDF inertias

Component	Mass (kg)
Base	0.150
Turntable	0.030
Link 1	0.095
Link 2	0.095
Gripper parameter	0.080
Each finger	0.010

For a uniform box of side lengths (a, b, c) , the URDF uses

$$I_{xx} = \frac{m}{12}(b^2 + c^2), \quad I_{yy} = \frac{m}{12}(a^2 + c^2), \quad I_{zz} = \frac{m}{12}(a^2 + b^2).$$

For a solid cylinder of radius r and height h ,

$$I_{xx} = I_{yy} = \frac{m}{12}(3r^2 + h^2), \quad I_{zz} = \frac{1}{2}mr^2.$$

These URDF values describe individual visual links. The compact PID plant uses separate effective diagonal joint inertias listed in Section 4.

1.4 Limits and command space

The URDF permits $q_2, q_3, q_4 \in [-90^\circ, 90^\circ]$. Hardware IK applies stricter servo-safe limits in its angle-from-horizontal convention:

$$\begin{aligned} 10^\circ &\leq \alpha_2 \leq 90^\circ, \\ -80^\circ &\leq \alpha_3 \leq 80^\circ, \\ -80^\circ &\leq \alpha_4 \leq 80^\circ. \end{aligned}$$

The deployment vector on `/arm_command` is

$$\mathbf{a} = \begin{bmatrix} u_b & \alpha_2 & \alpha_3 & \alpha_4 & c_g \end{bmatrix}^T,$$

where $u_b \in [-1, 1]$ is normalized base velocity and $c_g \in [0, 1]$ is normalized gripper closure (0 open, 1 closed).

2 Coordinate conventions and forward kinematics

2.1 Hardware-to-URDF conversion

Hardware IK measures arm angles from the horizontal plane, while the URDF measures pitch from vertical. The exact conversion implemented by the simulator is

$$\boxed{q_2 = \frac{\pi}{2} - \alpha_2, \quad q_3 = -\alpha_3, \quad q_4 = -\alpha_4.} \quad (1)$$

This conversion has been checked by substituting solved configurations into the URDF forward kinematics; tested Cartesian residuals were below displayed floating-point precision (0.000 mm).

2.2 Planar forward kinematics

Define cumulative pitch angles

$$\theta_2 = q_2, \quad \theta_3 = q_2 + q_3, \quad \theta_4 = q_2 + q_3 + q_4.$$

The radial reach and tip height are

$$r(\mathbf{q}) = L_1 \sin \theta_2 + L_2 \sin \theta_3 + L_3 \sin \theta_4, \quad (2)$$

$$z(\mathbf{q}) = h_0 + L_1 \cos \theta_2 + L_2 \cos \theta_3 + L_3 \cos \theta_4. \quad (3)$$

Base yaw revolves the planar chain around the world z axis:

$$\boxed{x = r \cos q_1, \quad y = r \sin q_1, \quad z = z(\mathbf{q}).} \quad (4)$$

Thus the Cartesian target is decomposed cylindrically as

$$q_1^* = \text{atan2}(y^*, x^*), \quad r^* = \sqrt{(x^*)^2 + (y^*)^2}.$$

3 Inverse kinematics

The deployed IK is solved in the vertical arm plane using hardware angles α_i . For a candidate end-effector approach angle ϕ , subtract link L_3 to obtain the wrist point relative to the shoulder:

$$w_x = r^\star - L_3 \cos \phi, \quad (5)$$

$$w_y = z^\star - h_0 - L_3 \sin \phi, \quad (6)$$

$$d^2 = w_x^2 + w_y^2. \quad (7)$$

A two-link solution exists only when

$$|L_1 - L_2| \leq d \leq L_1 + L_2.$$

The elbow angle follows the cosine law:

$$\cos \alpha_3 = \frac{d^2 - L_1^2 - L_2^2}{2L_1L_2}. \quad (8)$$

Both branches are evaluated:

$$\alpha_3 = \pm \arccos \left(\frac{d^2 - L_1^2 - L_2^2}{2L_1L_2} \right).$$

Then

$$k_1 = L_1 + L_2 \cos \alpha_3, \quad (9)$$

$$k_2 = L_2 \sin \alpha_3, \quad (10)$$

$$\alpha_2 = \text{atan2}(w_y, w_x) - \text{atan2}(k_2, k_1), \quad (11)$$

$$\alpha_4 = \phi - (\alpha_2 + \alpha_3). \quad (12)$$

The solver tests $\phi = -90^\circ$ first, then searches outward in 1° increments through the full angular range. It rejects configurations violating servo-safe limits or $z < 0$. The primary cost minimizes departure from a downward-pointing end effector:

$$J_\phi = |\phi + 90^\circ|.$$

Among equally oriented candidates, continuity is selected by

$$J_q = (\alpha_2 - \alpha_{2,p})^2 + (\alpha_3 - \alpha_{3,p})^2 + (\alpha_4 - \alpha_{4,p})^2.$$

A 10^{-6} rad comparison tolerance preserves mathematically on-limit poses; accepted outputs are clamped back to the hard limits, so tolerance never widens the actuator command space.

4 Numerical dynamic plant

4.1 Model structure

A general rigid manipulator obeys

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}(\mathbf{q}) + \mathbf{F}(\dot{\mathbf{q}}) = \boldsymbol{\tau} + \boldsymbol{\tau}_d. \quad (13)$$

The implemented numerical plant intentionally approximates this with diagonal constant inertia, viscous damping, exact configuration-dependent gravity for lumped masses, and an external disturbance input:

$$\boxed{I_i \ddot{q}_i = \tau_i - b_i \dot{q}_i - G_i(\mathbf{q}) + \tau_{d,i}.} \quad (14)$$

Coriolis coupling, Coulomb friction, backlash, gearbox elasticity, electrical motor dynamics, and voltage sag are not modeled.

These are control-development values, not identified servo-shaft inertias or measured torque limits.

Table 2: Nominal simulation parameters.

Joint	I_i (kg m ²)	b_i	$ \tau_i _{\max}$
J1	0.0012	0.003	2.2
J2	0.0100	0.025	4.8
J3	0.0038	0.012	2.2
J4	0.0008	0.005	2.2

4.2 Gravity model

Let $m_1 = m_2 = 0.095$ kg, $m_g = 0.080$ kg, $m_f = 0.010$ kg, and define the distal lumped mass

$$m_d = m_g + 2m_f = 0.100 \text{ kg}.$$

With $s_2 = \sin q_2$, $s_{23} = \sin(q_2 + q_3)$ and $s_{234} = \sin(q_2 + q_3 + q_4)$, the implemented gravity vector is

$$G_1 = 0, \quad (15)$$

$$G_2 = -g \left[\frac{m_1 L_1}{2} s_2 + m_2 \left(L_1 s_2 + \frac{L_2}{2} s_{23} \right) + m_d \left(L_1 s_2 + L_2 s_{23} + \frac{L_3}{2} s_{234} \right) \right], \quad (16)$$

$$G_3 = -g \left[\frac{m_2 L_2}{2} s_{23} + m_d \left(L_2 s_{23} + \frac{L_3}{2} s_{234} \right) \right], \quad (17)$$

$$G_4 = -g \left[\frac{m_d L_3}{2} s_{234} \right], \quad (18)$$

with $g = 9.80665$ m/s². J1 has no gravity moment because its axis is vertical in this model.

4.3 Discrete integration and stops

The controller and plant run at

$$\Delta t = 0.005 \text{ s} \quad (200 \text{ Hz}).$$

Semi-explicit Euler integration is used:

$$\dot{q}_{i,k+1} = \dot{q}_{i,k} + \ddot{q}_{i,k} \Delta t, \quad (19)$$

$$q_{i,k+1} = q_{i,k} + \dot{q}_{i,k+1} \Delta t. \quad (20)$$

Measured state is published at 50 Hz. J2–J4 have ideal hard stops at $\pm\pi/2$; outward velocity is cancelled at a stop while inward motion remains possible. J1 is continuous.

5 PID controller derivation

5.1 Position loops for J2–J4

For joint i , define

$$e_i = q_i^* - q_i.$$

The implemented controller differentiates the measurement rather than the error:

$$\tau_{fb,i} = K_{p,i} e_i + K_{i,i} \eta_i - K_{d,i} \dot{q}_i. \quad (21)$$

This prevents a setpoint step from producing derivative kick. Gravity is fed forward and total effort is saturated:

$$\tau_i = \text{sat}_{[-\tau_{\max,i}, \tau_{\max,i}]}(\tau_{fb,i} + G_i(\mathbf{q})). \quad (22)$$

If the model were exact and the arm stationary at the target, $e_i = 0$ and $\tau_i = G_i$, which cancels gravity in Equation (14).

5.2 Discrete integral and anti-windup

The candidate integral state is

$$\tilde{\eta}_k = \text{sat}_{[-\eta_{\max}, \eta_{\max}]}(\eta_{k-1} + e_k \Delta t).$$

It is accepted if the raw feedback command is unsaturated, or if the error points back toward the controllable range. Otherwise $\eta_k = \eta_{k-1}$. This conditional integration prevents continued windup while an actuator is pinned. Position-loop integral limit is 2.0 rad s.

5.3 Base velocity loop

The base command remains compatible with the continuous-rotation servo:

$$\dot{q}_1^* = 1.5 \text{ sat}_{[-1,1]}(u_b).$$

Velocity error is

$$e_v = \dot{q}_1^* - \dot{q}_1,$$

and the velocity PID is

$$\tau_1 = K_{p,v} e_v + K_{i,v} \int e_v dt - K_{d,v} \ddot{q}_1.$$

Raw measured acceleration and its filtered derivative state are

$$\ddot{q}_{1,k}^{raw} = \frac{\dot{q}_{1,k} - \dot{q}_{1,k-1}}{\Delta t}, \quad (23)$$

$$\ddot{q}_{1,k}^{(m)} = \lambda \ddot{q}_{1,k-1}^{(m)} + (1 - \lambda) \ddot{q}_{1,k}^{raw}, \quad \lambda = 0.95. \quad (24)$$

The simulated effort is limited to ± 2.2 . The low-pass derivative is essential: differentiating sampled encoder velocity directly amplifies noise and, in a discrete loop, can destabilize the base controller.

5.4 Gripper loop

Let $c_g \in [0, 1]$ be desired closure and $c \in [0, 1]$ be measured simulated closure. A position PID drives the normalized plant. The corresponding finger joint coordinate is

$$q_f = -0.03(1 - c). \quad (25)$$

Thus $c = 0$ gives $q_f = -0.03$ m and 90 mm measured finger separation (open), while $c = 1$ gives $q_f = 0$ and 30 mm separation (closed).

5.5 Controller gains

Loop	K_p	K_i	K_d
J1 velocity	0.035	0.080	0.0015
J2 position	0.90	0.22	0.15
J3 position	0.42	0.12	0.075
J4 position	0.16	0.045	0.030
Gripper closure	18.0	5.0	0.8

These gains are tuned only for the nominal plant in this document. Dimensional units depend on each simulated effort channel; they are not automatically valid PWM gains.

6 Closed-loop interpretation

Ignoring gravity feedforward error, integral action, and saturation, a single position axis has

$$I\ddot{q} + b\dot{q} = K_p(q^* - q) - K_d\dot{q}.$$

The closed-loop transfer function is

$$\frac{Q(s)}{Q^*(s)} = \frac{K_p}{Is^2 + (b + K_d)s + K_p}. \quad (26)$$

Equivalent second-order parameters are

$$\omega_n = \sqrt{\frac{K_p}{I}}, \quad \zeta = \frac{b + K_d}{2\sqrt{IK_p}}.$$

Integral action adds a pole at the origin and removes constant residual error, but reduces phase margin. Saturation, gravity variation, and the hard stops make the actual model nonlinear, so these formulas guide tuning but do not replace simulation or measurement.

7 ROS 2 signal model

Topic	Type	Meaning
/arm_command	Float32MultiArray	Deployment-format setpoint $[u_b, \alpha_2, \alpha_3, \alpha_4, c_g]$.
/joint_states	JointState	Simulated encoder positions, velocities, and efforts; this is measured plant state, not a copied command.
/joint_setpoint	JointState	Desired URDF position and base velocity.
/pid_diagnostics	Float32MultiArray	Five errors followed by five controller efforts.
/sim_disturbance	Float32MultiArray	Persistent additive efforts $[\tau_{d1}, \tau_{d2}, \tau_{d3}, \tau_{d4}, d_g]$; publish zeros to remove.

The full simulation launches with:

```
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 launch manipulator_control pid_sim.launch.py
```

This launch runs Qt, `arm_commander`, the PID plant, click-to-target, `robot_state_publisher`, and RViz. It intentionally does not run `ik_node`, because two publishers on `/joint_states` would make the measured state nondeterministic.

8 Numerical validation results

For target $(x, y, z) = (0.20, 0.00, 0.02)$ m, measured behavior was: A 0.25 N m shoulder disturbance applied for 0.5 s produced a 15.33° peak error and recovered to 0.614° after 4 s. Base command $u_b = 0.5$ requested 0.750 rad/s, measured 0.750 rad/s, and stopped at 0.0001 rad/s. The gripper reached both exact endpoints.

Joint	Final error	Peak speed	Settle within 1°
J1	0.000°	0.000 rad/s	immediate
J2	0.488°	2.733 rad/s	3.210 s
J3	0.199°	1.128 rad/s	0.429 s
J4	0.003°	0.019 rad/s	immediate

An extreme ± 20 N m stress input—several times the modeled actuator limits—could not move J2–J4 beyond $\pm 90^\circ$ and did not produce NaN or infinite state. This validates stop invariants, not realistic recovery.

For $(x, y, z) = (0.22, 0.08, 0.04)$ m after settling:

$$\mathbf{q}_{2:4}^* = \begin{bmatrix} 0.79758 & 0.53283 & 1.32249 \end{bmatrix}^T \text{ rad},$$

and measured values matched to the displayed precision. RViz rendered the measured pose through `robot_state_publisher`.

9 Encoder-based hardware deployment

9.1 Required measurements

Each encoder sample should provide timestamped q_i . Velocity should be estimated with a filtered differentiator; a simple first-order form is

$$\dot{q}_{raw,k} = \frac{\text{wrap}(q_k - q_{k-1})}{\Delta t}, \quad (27)$$

$$\dot{q}_k = \lambda \dot{q}_{k-1} + (1 - \lambda) \dot{q}_{raw,k}, \quad (28)$$

with angle wrapping for continuous J1. Choose λ from measured encoder noise and sample rate. Publish hardware measurements as `/joint_states`.

9.2 Controller migration

Preserve the current setpoint and diagnostic interfaces. Replace only the numerical plant step with encoder input and actuator output:

$$\text{simulation: } \tau \rightarrow \text{numerical plant} \rightarrow (q, \dot{q}),$$

$$\text{hardware: } \tau \rightarrow \text{PWM/current command} \rightarrow \text{encoder } (q, \dot{q}).$$

There must be exactly one actuator-command owner. Do not run the PID hardware node and an open-loop ESP32 command publisher simultaneously.

9.3 Important limitation of hobby positional servos

Shoulder, elbow, wrist, and gripper servos already contain hidden inner position controllers. An outer ROS PID therefore forms a cascade:

$$\text{outer PID} \rightarrow \text{servo angle command} \rightarrow \text{internal controller} \rightarrow \text{motor}.$$

The outer loop should initially be slow relative to the inner loop. Start with trajectory limiting and P-only correction, then add D and finally I only when measurements justify them. Aggressive outer gains can excite the unknown inner loop, backlash, and compliance.

For the continuous base with an encoder, use a cascaded controller. Outer yaw position generates velocity:

$$\dot{q}_1^* = \text{sat}_{[-\omega_{max}, \omega_{max}]}(K_{p,\theta} \text{wrap}(q_1^* - q_1)),$$

and the inner velocity PID generates the normalized servo command. Wrapping is essential near $\pm\pi$:

$$\text{wrap}(\delta) = \text{atan2}(\sin \delta, \cos \delta).$$

9.4 Identification and tuning sequence

1. Calibrate encoder zero, sign, wrap, sample timing, and missing-data behavior.
2. Measure J1 neutral pulse and velocity versus PWM offset in both directions.
3. Apply small safe steps and log setpoint, encoder position, velocity, and PWM.
4. Estimate delay, deadband, steady speed, damping, and gravity/load dependence.
5. Replace nominal I_i, b_i and actuator limits with identified values.
6. Tune P first at low payload; increase until responsive but non-oscillatory.
7. Add D using filtered measured velocity to control overshoot.
8. Add small I only to remove persistent bias; retain anti-windup.
9. Validate at joint limits, supply-voltage extremes, maximum payload, and stale feedback.
10. Require a hardware watchdog that stops J1 and freezes safe position commands when encoder or ROS commands are stale.

10 Model limitations and interpretation

- The diagonal inertia approximation neglects configuration-dependent mass matrix and Coriolis coupling.
- Centers of mass are idealized at link midpoints; wiring, servo bodies, horns, fasteners, and payload are not identified.
- Effort units are numerical torque-like units; hobby-servo PWM is not torque.
- Gravity feedforward uses the same approximate model as the plant, so its nominal steady-state result is intentionally optimistic.
- The 1-degree IK orientation search can make workspace feasibility granular near boundaries.
- There is no collision, table contact, Coulomb friction, stiction, backlash, thermal limit, current limit, voltage sag, or communications jitter model.
- Numerical gains demonstrate controller structure and behavior. They are not safe hardware gains without encoder calibration and plant identification.

11 Source-to-equation traceability

Source	Mathematical responsibility
<code>arm_ik_2d.py</code>	Geometry, limits, cylindrical conversion, FK/IK, approach-angle search, continuity selection.
<code>pid_sim_node.py: PID.update</code>	Equation (21), integral clamp, derivative-on-measurement, conditional anti-windup.
<code>pid_sim_node.py: gravity_torque</code>	Implemented $\mathbf{G}(\mathbf{q})$.
<code>pid_sim_node.py: timer_cb</code>	Equations (22) and (14), Euler integration, saturation, hard stops.
<code>manipulator.urdf.xacro</code>	Link dimensions, masses, link inertias, axes, and joint limits.
<code>arm_commander.py</code>	Target-to-IK conversion and 50 Hz deployment vector.
<code>pid_sim.launch.py</code>	Single-owner closed-loop simulation topology.

Conclusion

The implemented simulator is a useful intermediate step between direct kinematic visualization and encoder-controlled hardware. Its central invariant is that setpoint and measurement are separate: PID effort changes the plant, and only plant state becomes `/joint_states`. The equations, numerical constants, limitations, and ROS interfaces are explicit, so future encoder data can replace nominal assumptions without changing the surrounding command stack.